

DS216: Machine Learning for Data Science

Assignment 1 Report

Kritika Gupta(25886)

February 28, 2026

Contents

1	Linear Discriminant Analysis (LDA)	2
1.1	The Homoscedastic Case	2
1.2	Part 2: The Heteroscedastic Case	3
1.3	Performance Gap LDA vs. QDA (Heteroscedastic case):	4
2	Image Segmentation via K-Means and GMM	4
2.1	Method for optimal Number of Clusters (k)	4
2.2	Initialization Comparison	4
3	Logistic Regression and SVM (SMO)	6
3.1	Data Preprocessing and PCA	6
3.2	Logistic Regression	7
3.3	SVM via SMO Optimization	7
3.4	Overall Comparison	7

1 Linear Discriminant Analysis (LDA)

1.1 The Homoscedastic Case

A 3-class, 10-dimensional dataset with each class having 500 samples was generated. The dataset was generated using the same feature covariance matrix Σ for each class. 70% of the data was used to find the LDA projection (eigenvectors), and the same was used to train both linear and quadratic classifiers.

Observations:

- Projected 2D data obtained using the computed within-class scatter (S_W) and between-class scatter (S_B) matrices, is well-separated and has similarly shaped clusters.
- Because the underlying data distributions share the same orientation and scale, all the classifiers have very similar performance with high accuracy. Results are reported below in Table 1.
- Note:- Reported QDA classifier accuracy is on 10D data, while for plotting decision boundaries, it is retrained on 2D data.

Table 1: Classifier Performance Comparison (Homoscedastic Data)

Classifier	Training Accuracy	Test Accuracy
Linear Classifier	0.98524	0.99111
KNN Classifier	0.98538	0.98889
QDA Classifier	0.9619	0.98889

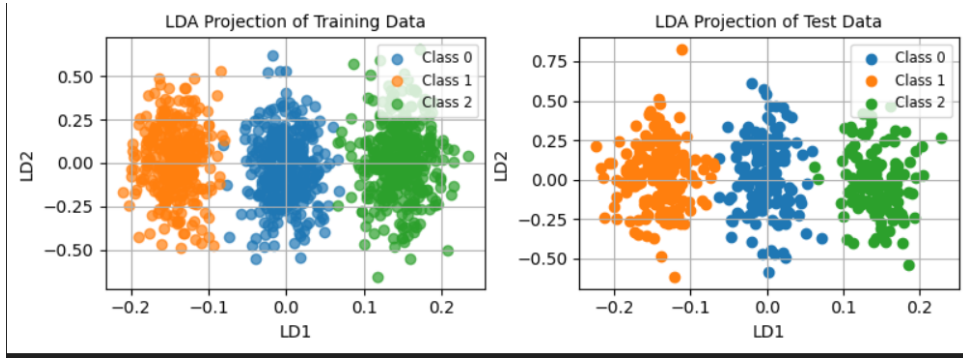


Figure 1: LDA 2D Projection of Homoscedastic Data

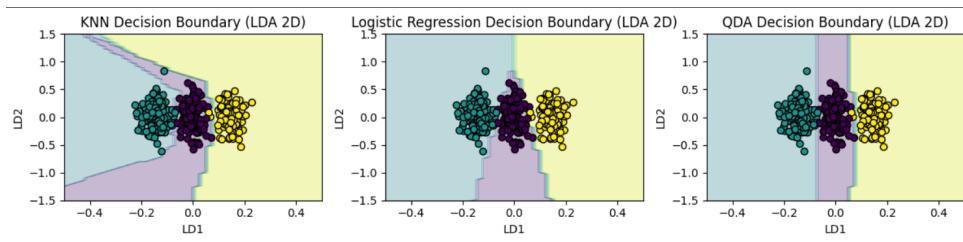


Figure 2: LDA 2D Decision Boundary of Homoscedastic Data

1.2 Part 2: The Heteroscedastic Case

A 3-class, 10-dimensional dataset with each class having 500 samples was generated. The dataset was generated using the different feature covariance matrix ($\Sigma_1, \Sigma_2, \Sigma_3$) for given classes. 70% of the data was used to find the LDA projection (eigenvectors), and the same was used to train both linear and quadratic classifiers.

Observations:

- Projected 2D data obtained using the computed within-class scatter (S_W) and between-class scatter (S_B) matrices, has different cluster shapes and orientations. Due to the different covariance matrix, a single within-class scatter matrix S_W fails to capture the true intra-class variance, and hence, no true separability is observed (i.e. overlapping classes)
- Because the underlying data distributions have different orientations and scales, violating the core assumption of standard LDA, accuracy drops for KNN and Logistic Regression. Results are reported below in Table 2.
- Note:- Reported QDA classifier accuracy is on 10D data, while for plotting decision boundaries, it is retrained on 2D data.

Table 2: Classifier Performance Comparison (Heteroscedastic Data)

Classifier	Training Accuracy	Test Accuracy
Linear Classifier	0.684762	0.715556
KNN Classifier	0.759048	0.664444
QDA Classifier	0.99619	1.0

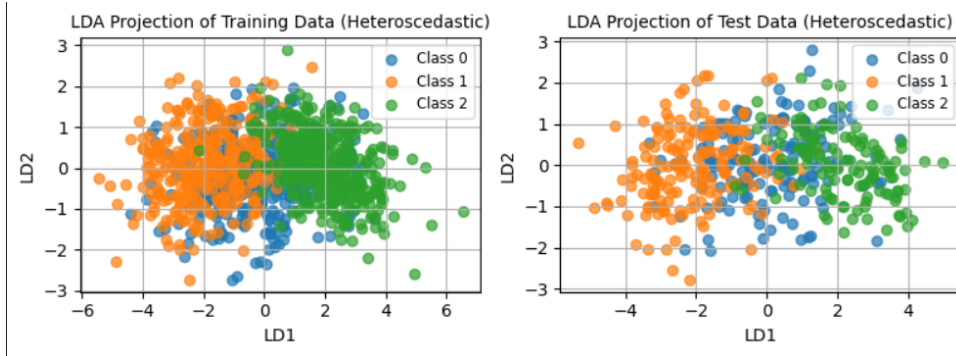


Figure 3: LDA 2D Projection of Heteroscedastic Data

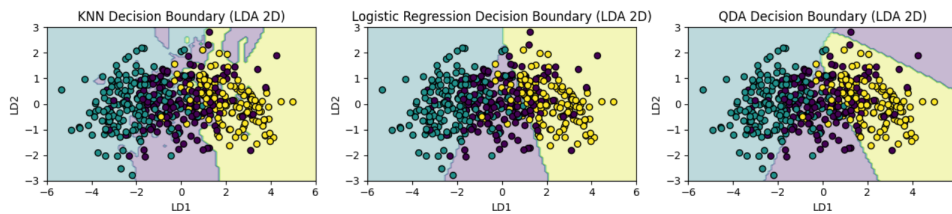


Figure 4: LDA 2D Decision Boundary of Heteroscedastic Data

1.3 Performance Gap LDA vs. QDA (Heteroscedastic case):

Applying linear and quadratic classifiers to the heteroscedastic data shows a significant performance gap. Quadratic Discriminant Analysis (QDA) outperforms linear models (like logistic regression) in terms of accuracy. Since QDA calculates a distinct covariance matrix for each class, it accurately models the varying densities and elliptical shapes of the clusters in the feature space, reducing misclassifications.

Decision Boundaries and Projections:

- **LDA with Logistic Regression:** This constructs linear decision boundaries (hyperplanes). In the 2D projected space, separating the classes. Because the data has varying covariance matrices and hence overlapping clusters, these straight lines cannot optimally separate clusters that exhibit different variances.
- **LDA with KNN:** Even KNN doesn't perform well with irregular boundaries due to its sensitivity to the irregular cluster shapes, hyperparameter K , and overlapping data and boundaries created by LDA's suboptimal projection in the heteroscedastic case.
- **QDA:** Constructs quadratic decision boundaries (conic sections like ellipses, parabolas, or hyperbolas). By using the differing covariance matrices for each class, QDA naturally wraps its decision boundaries around the distinct shapes of the class distributions, effectively reducing the overlap.

2 Image Segmentation via K-Means and GMM

2.1 Method for optimal Number of Clusters (k)

The following methods were used to determine the optimal number of clusters for image segmentation:

- **K-Means (Elbow Method):** The Within-Cluster Sum of Squares (WCSS) was plotted against various values of k , then found the "elbow" point—the value of k where the second-order difference of WCSS is maximized. This point represents where increasing k won't get much reduction in variance (WCSS).
- **GMM (AIC/BIC):** For the Gaussian Mixture Model, the Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) were evaluated. Both metrics penalize model complexity. In general, the optimal k is selected by finding the minimum AIC/BIC value, but in this case, the dataset was quite large and the $\log N$ term dominated the AIC/BIC values, and no clear minima was observed for the given k -values; hence, the same elbow method was used to find the optimal k from AIC/BIC graphs.

2.2 Initialization Comparison

The optimal k was used to segment and reconstruct the images using two initializations: random and K-Means++.

Observations:

- **Convergence Speed:** K-Means++ initialization significantly reduces the number of iterations required to converge for both algorithms. By intelligently choosing the initial centroids such that no two centroids occur too close, K-Means++ avoids poor local minima and requires fewer updates to stabilize.
- **Image Quality (PSNR):** K-means++ initialization has slightly better values for PSNR than the random one. However, K-means has consistently better values for PSNR than the GMM clustering model.
- **Image Quality (SSIM):** K-means++ initialization has slightly better values for SSIM than the random one. However, K-means has consistently better values for SSIM than the GMM clustering model.
- **General:** Random initialization results in inconsistent results, while K-means++ gives almost the same results across multiple runs. From the reconstructed image, one can conclude that GMM is more affected by random initialization than the K-means method. Moreover, GMM takes more iteration in general than K-means, making K-means computationally better.
- Below is the table summarizing results for a few images (3) and their respective reconstructed images (5 and 6):

Table 3: K-Means and GMM Performance Comparison

Image	Run	Method	Initialization	Iterations	PSNR	SSIM
1	1	K-Means	Random	81	22.8673	0.7724
1	1	K-Means	K-Means++	81	23.1486	0.82196
1	1	GMM	Random	99	19.882	0.63943
1	1	GMM	K-Means++	99	19.8787	0.63957
1	2	K-Means	Random	73	23.1448	0.80191
1	2	K-Means	K-Means++	73	23.1448	0.80191
1	2	GMM	Random	99	19.8889	0.631002
1	2	GMM	K-Means++	99	19.8871	0.630819
2	1	K-Means	Random	81	21.7912	0.612631
2	1	K-Means	K-Means++	81	21.7912	0.612631
2	1	GMM	Random	56	20.171	0.569468
2	1	GMM	K-Means++	73	20.171	0.569468
2	2	K-Means	Random	99	21.7912	0.61261
2	2	K-Means	K-Means++	99	21.7912	0.612631
2	2	GMM	Random	99	19.8889	0.583549
2	2	GMM	K-Means++	75	19.8871	0.569468

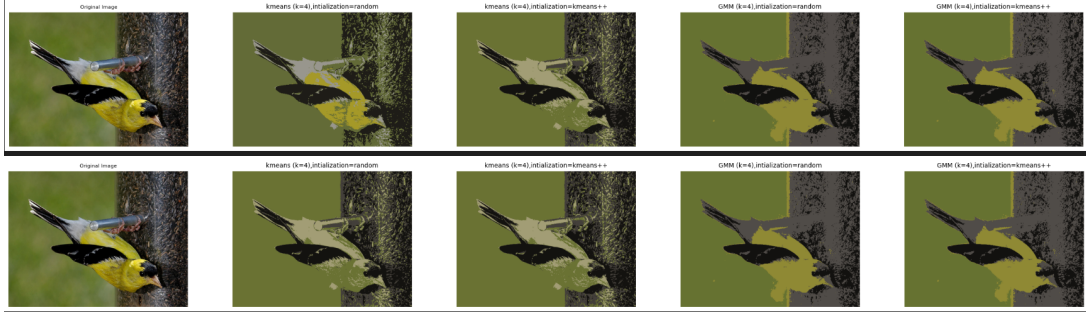


Figure 5: Original and Reconstructed Images using K-Means and GMM



Figure 6: Original and Reconstructed Images using K-Means and GMM

3 Logistic Regression and SVM (SMO)

3.1 Data Preprocessing and PCA

The Breast Cancer Wisconsin dataset had all features in multiple ranges, so it was preprocessed by standardizing using Z-score normalization so each feature contributes equally to the PCA and won't be dominated by the feature with a large range of values. Then Principal Component Analysis (PCA) was applied to observe the effect of dimensionality reduction (using different numbers of dimensions).

Observation: Even the first few dimensions retain the vast majority of the variance ($> 85\%$) as shown in Table4. This reduction drastically speeds up the convergence of both Logistic Regression and the SMO SVM without heavily degrading classification performance.

Table 4: PCA dimensions and variance retained

PCA Dim	Variance Retained (%)
2	63.2561
5	85.1043
10	95.5019
15	98.6521
30	100

3.2 Logistic Regression

A custom Logistic Regression model was implemented with gradient descent and L_2 regularization. The model efficiently optimizes the log-loss and achieves a robust baseline accuracy. The L_2 regularization parameter (λ) successfully penalizes large weight updates, keeping the accuracy and other metrics stable.

3.3 SVM via SMO Optimization

The Support Vector Machine was implemented using the Sequential Minimal Optimization (SMO) algorithm. Three kernels were evaluated:

- **Linear Kernel:** Computes the inner product $K(x_i, x_j) = x_i^T x_j$. It gives a linear decision boundary similar to logistic regression. It gave high accuracy, showing that the breast cancer dataset is largely linearly separable in the high-dimensional space.
- **Polynomial Kernel:** $K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$. Introduced non-linearity but was observed to be highly sensitive to the degree d . It exhibited longer convergence times and very low accuracy for lower PCA dimensions while gave highest accuracy and F-1 score for higher PCA dimensions.
- **RBF (Gaussian) Kernel:** $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$. It implicitly maps features into an infinite-dimensional space. It captures complex, non-linear relationships. But its performance peaks at 5 dimensions, and its performance saturates at higher dimensions, showing that it might be overfitting in higher dimensions.

3.4 Overall Comparison

- **Convergence:** SVM required significantly more computational time than standard Logistic Regression gradient descent, especially for the polynomial and RBF kernels due to computing pairwise distances. However, applying reducing dimensions prior to SVM training reduced this computational overhead.
- **Performance:** Logistic Regression gave constantly high accuracy across all PCA dimensions, and the same was observed with SVM linear kernel, which shows that the given data set is highly separable and a linear decision boundary can be found easily. While accuracy fluctuates for RBF kernel, showing that it might be overfitting to the data in higher dimensions, polynomial is highly dependent on dimensions and degree used to train the model.
- Below is the table 5 and graph 7 summarising the results -

Table 5: PCA—different dimension Performance Comparison

PCA Dim	Model	Accuracy	Precision	Recall	F1-Score
2	Logistic Regression	0.918605	0.878788	0.90625	0.892308
5	Logistic Regression	0.930233	0.882353	0.9375	0.909091
10	Logistic Regression	0.930233	0.882353	0.9375	0.909091
15	Logistic Regression	0.930233	0.882353	0.9375	0.909091
30	Logistic Regression	0.94186	0.909091	0.9375	0.923077
2	linear	0.930233	0.882353	0.9375	0.909091
5	linear	0.930233	0.882353	0.9375	0.909091
10	linear	0.94186	0.909091	0.9375	0.923077
15	linear	0.953488	0.9375	0.9375	0.9375
30	linear	0.94186	0.935484	0.90625	0.920635
2	polynomial	0.604651	0.483871	0.9375	0.638298
5	polynomial	0.930233	0.861111	0.96875	0.911765
10	polynomial	0.930233	0.90625	0.90625	0.90625
15	polynomial	0.94186	0.909091	0.9375	0.923077
30	polynomial	0.965116	0.967742	0.9375	0.952381
2	rbf	0.906977	0.852941	0.90625	0.878788
5	rbf	0.953488	0.9375	0.9375	0.9375
10	rbf	0.918605	0.857143	0.9375	0.895522
15	rbf	0.895349	0.828571	0.90625	0.865672
30	rbf	0.895349	0.828571	0.90625	0.865672

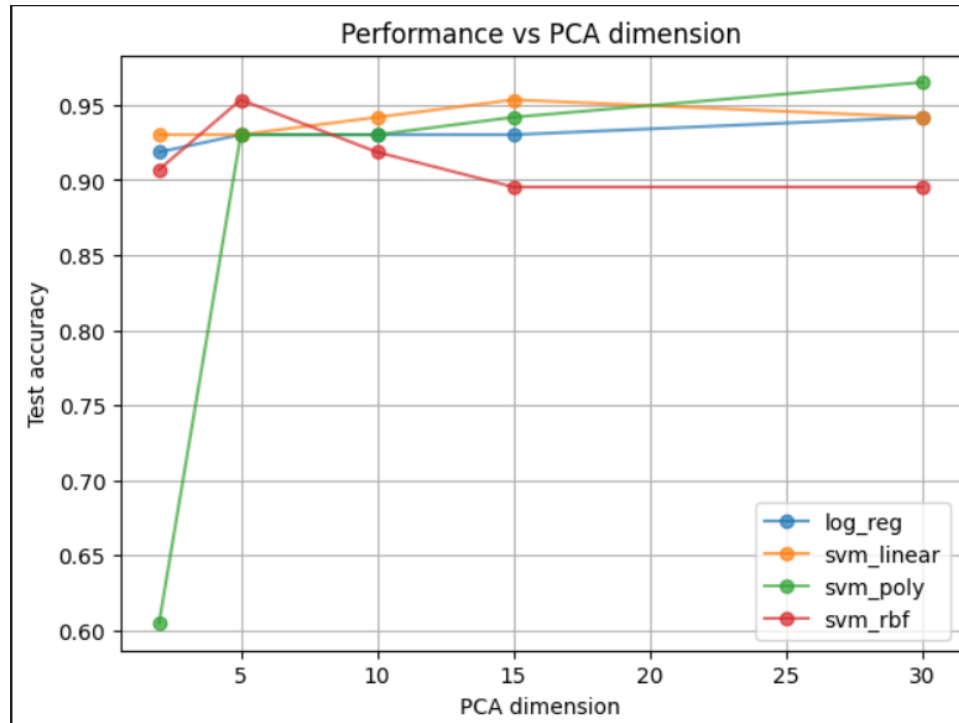


Figure 7: Accuracy across PCA Dimensions and Models